

班级心理学项目：梦境数智化平台 —— 团队启动手册

项目名称：LucidMind 项目周期：2025.12 - 2026.05

第一部分：给各组的一封信

亲爱的同学们，

欢迎加入这个充满想象力和前沿技术的心理学 AI 项目！我们不是在写课后作业，而是在共同打造一个**真正能够帮助人、并拥有巨大商业潜力的数智化产品**。

我知道大家可能学到了一些 Java 和前端基础，但面对“AI 心理学”这样宏大的主题，可能会有些许焦虑。请相信，这正是你们将课堂知识转化为实战经验的最佳机会。我们会一起把这栋“梦想之屋”盖起来！

1. 给前端组

- **你们是谁：**你们是整个平台的**用户体验设计师和魔法师**。一个好的前端，能让用户忘记背后的代码，沉浸在我们创造的“梦幻”世界里。
- **你们的知识起点：**已掌握 HTML、CSS 和 **JavaScript 基础**。
- **你们做什么：**
 - **核心职责：**将后端（数据）和算法（智慧）巧妙地结合起来，以**美观、流畅、响应式**的方式呈现。你们要用颜色、动画、排版来营造出治愈且私密的梦境日记氛围。
 - **深度拓展：**既然大家已经有了 JS 基础，我们就要跳出静态页面，实现复杂的交互逻辑。比如，如何让用户在输入梦境时，页面能实时做出梦幻的光影反馈？如何用 JS 优化数据提交的效率？
- **技术栈：**HTML, CSS, JavaScript, 响应式设计, **Vue / React (主流框架)**。
- **就业与薪资展望：**
 - **职业方向：**前端开发工程师、UI/UX 工程师。在未来，前端正逐渐向“**前端工程化**”和“**可视化编程**”转型。掌握主流框架的同学，可以轻松进入各类互联网公司、SaaS (软件即服务) 公司。
 - **薪资潜力：**市场对能独立完成复杂交互、注重用户体验 (UX) 的前端人才求贤若渴。这是一个就业岗位数量多、起薪高，且能快速拿到大厂 Offer 的赛道。

2. 给后端组

- **你们是谁：**你们是平台的**安全卫士、交通枢纽和数据管家**。虽然不直接与用户见面，但你们决定了平台能否稳定、可靠、安全地运行。
- **你们的知识起点：**已掌握 Java 基础，特别是**输入输出流 (IO Stream)** 的处理。
- **你们做什么：**
 - **核心职责：**设计数据库结构，管理用户身份认证和权限。你们的工作是建立一套**高效、可靠的 API 接口**，确保前端发来的梦境数据能安全、有序地存入数据库，并能被算法组高效取用。
 - **深度拓展：**你们的 IO 流知识将是处理网络数据和文件读写的基础。前端和算法组的每次请求和响应，本质上都是网络上的 IO 操作。你们需要学习如何用 **Spring Boot** 这样的企业级框架，优雅地封装这些数据流，保证 API 接口的高性能和安全性。
- **技术栈：**Java, **Spring Boot**, MySQL (或 PostgreSQL), RESTful API 设计。
- **就业与薪资展望：**

- **职业方向**：Java 后端开发工程师、系统架构师、数据工程师。
- **薪资潜力**：Java 工程师一直是IT行业的“硬通货”，应用领域极其广泛（金融、电商、云计算等）。掌握 Spring Cloud/Spring Boot 全家桶后，职业天花板很高，未来可以成长为支撑百万级用户系统的架构师。

3. 给算法组

- **你们是谁**：你们是赋予这个应用“灵魂”和“智慧”的人。你们的工作是让冰冷的机器，学会理解人类深奥的潜意识。
- **你们的知识起点**：已了解部分 Java/前端知识，并决定采用 **Coze Workflow** 方案。
- **你们做什么**：
 - **核心职责**：通过 **Coze Workflow**，将复杂的 LLM 能力编排成一个个智能体（Agent）。例如，一个智能体负责“情绪分析”，另一个负责“关键词提取”，最后通过工作流串联起来，实现精准的“AI 解梦”。这让我们可以专注于“**Prompt 设计**”和“**AI 逻辑**”，而不是模型部署。
 - **深度拓展**：你们正在实践 NLP 领域最前沿的应用。例如，**Cortal 等人 (2024)** 的研究展示了 LLMs 如何实现梦境叙事中角色和情绪的自动检测；**Bertolini 等人 (2024)** 证明了 LLM 甚至可以达到与人类心理学专家相似的情绪编码准确度。你们的工作就是将这些研究成果通过 Coze 工作流落地。
- **技术栈**：**Coze Workflow**, Prompt Engineering (提示词工程), API 接口设计, (长期：Python, LLM, NLP)。
- **就业与薪资展望**：
 - **职业方向**：AI 工程师、LLM 应用开发工程师、数据科学家。
 - **薪资潜力**：算法领域是目前技术岗位的**薪资天花板**。尤其是在 LLM 浪潮下，能够设计和优化 Prompt、构建多 Agent 系统的工程师，是各大科技公司和AI创新企业的核心竞争力。

4. 给测试组

- **你们是谁**：你们是系统的**质量保证（QA）工程师**。你们是产品发布前的最后一道防线，是团队中不可或缺的“**找茬专家**”。
- **你们的知识起点**：暂无系统学习测试知识。
- **你们做什么**：
 - **核心职责**：从零开始学习测试思维，保障代码逻辑和数据流的正确性。在第一周，你们就要学会使用 **Postman** 这个强大的工具，去验证后端哥哥姐姐们开发的 API 接口能否按预期工作。
 - **深度拓展**：测试绝不是简单的“点一点”。优秀的测试工程师需要具备**逆向思维**和**严密的逻辑**。你们要思考：如果用户输入了一个 5000 字的梦境，系统会不会崩溃？如果前端发来的数据格式不对，后端能不能优雅地报错？你们制定的测试用例，将是项目质量的黄金标准。
- **技术栈**：**Postman (接口测试)**, 测试用例设计 (黑盒测试), (未来：自动化测试框架)。
- **就业与薪资展望**：
 - **职业方向**：软件测试工程师、自动化测试工程师、质量管理专家。
 - **薪资潜力**：随着软件复杂度的增加，QA 的重要性越来越高。这是一个稳健的职业方向，特别是在金融、医疗等对质量要求极高的行业，资深测试工程师拥有深厚的领域知识和极高的议价能力。

第二部分：人工智能与心理学梦境分析的前沿趋势

(注：本部分内容结合了项目组提供的最新参考文献，可用于立项报告或答辩PPT)

1. 从手动编码到 AI 自动化

在传统的心理学研究中，梦境分析依赖于训练有素的专家进行耗时的人工编码。这不仅耗时，而且成本高昂。

- AI 的突破：**最新的研究表明，使用大型语言模型（LLMs）可以达到与人类专家相当的准确率。例如，**Bertolini 等人 (2024)** 的研究证明，LLM 能够准确检测梦境报告中的情绪（如愤怒、快乐），这为我们项目的“自动评分”功能提供了坚实的理论依据。我们的 AI 不仅能分析出梦见“老虎”代表恐惧，还能理解“追逐”这个动作背后的焦虑情绪。

2. 梦境的“可预测性”与数据驱动

很多人认为梦境是随机且杂乱无章的。但 **Bertolini (2023)** 的研究通过 GPT-2 模型分析发现，梦境文本实际上比维基百科的文本更具有“可预测性”。这意味着梦境有着潜在的结构和语法，非常适合用 AI 进行建模和趋势分析。

- 商业价值：**我们可以利用这一点，为用户生成“月度梦境情绪趋势图”，用数据告诉用户他们潜意识的规律，从数据层面进行自我觉察。

3. 生成式 AI 与沉浸式重温 (Immersive Reliving)

这是我们项目最具**创新性**的地方。我们不止于“分析”，更要走向“生成”和“干预”。

- 前沿案例：****Liu 等人 (2025)** 提出了 **DreamLLM-3D** 系统。他们利用 LLM 分析梦境实体（角色、物体），并结合生成式 AI 将其转化为 3D 点云模型，**甚至能根据梦境情绪改变 3D 模型的颜色**（例如愤怒是红色，悲伤是蓝色）。
- 我们的机会：**我们虽然不能一步到位做 VR，但我们可以参考 **Berger 等人 (2024)** 的研究，利用大模型根据用户的梦境特征和人口统计学信息来**生成**个性化的心理干预故事或“清醒梦”剧本，以辅助认知行为疗法（CBT）。

第三部分：项目长期规划 (2025.12 - 2026.05)

我们将整个开发周期分为三个阶段：**基础建设期**、**核心功能开发期**、**集成与优化期**。

阶段	时间	核心目标	重点事项
阶段一： 地基与原型	2025.12	全员入门 & 环境搭建	- 前端：画出静态的 HTML 页面（梦境日记页）。- 后端：跑通 Java Hello World，连上 MySQL。- 算法：跑通一个简单的 BERT 文本分类 Demo。
	2026.01	最小可行性产品 (MVP) v0.1	- 实现最简单的功能：用户能输入文字，后端能存，前端能显示“保存成功”。- 确定数据库表结构（用户表、日记表）。
阶段二： 核心功能	2026.02	心理量表 & 基础分析	- 前端：制作心理量表问卷页。- 后端：实现量表自动算分逻辑。- 算法：训练一个能识别“情绪”的简单模型。
	2026.03	AI 接入 & 智能体	- 攻坚战： 后端接入算法模型接口。- 实现“AI 解梦”功能（对接 LLM API）。- 测试组开始进行接口测试 (Postman)。

阶段	时间	核心目标	重点事项
阶段三： 整合交付	2026.04	数据可视化 & 联调	- 前端：引入 ECharts 画图表（情绪趋势）。- 前后端完全联调。- 尝试加入“生成式”功能（如根据梦境生成一张图）。
	2026.05	测试、部署 与展示	- 全面测试（找 Bug）。- 部署到云服务器。- 准备 PPT 和演示视频，强调我们的“数智化”创新点。

第四部分：第一周任务清单 (Kick-off Week)

【温馨提示】第一周是建立信心和打好地基的关键！任务会分解得很细，请大家一步一个脚印来完成。遇到任何环境配置或基础代码问题，都请及时在组内或大群中提问，我们一起解决！

1. 前端组 (Frontend)：构建梦境记录页面的 UI 骨架

- **任务 A (技术选型调研)：**
 - **目标：**为团队的未来技术方向做准备。
 - **步骤：**
 1. **收集信息：**搜索并阅读 Vue 和 React 这两个主流前端框架的**核心概念**（例如：组件化、单向/双向绑定等）和**各自的优势**（例如：Vue 易学，React 灵活）。
 2. **产出：**整理成一份简单的文字报告或幻灯片（2-3页），在下周的会议上向团队简要介绍你的发现。
- **任务 B (布局与样式 - 视觉魔法)：**
 - **目标：**用 HTML 和 CSS 完成梦境记录页面的静态原型，并初步实现**响应式设计**。
 - **步骤：**
 1. **HTML 骨架：**创建一个 `dream_record.html` 文件，包含以下核心元素：一个大标题（“梦境记录”）、一个大文本输入框（`<textarea>`），以及一个“保存”按钮（`<button>`）。给它们设置好容易识别的 ID 或 Class 名称。
 2. **CSS 样式：**利用 CSS 实现“梦幻紫”或“静谧蓝”的渐变背景。为输入框和按钮添加圆角和阴影效果，营造治愈感。
 3. **响应式 (RWD)：**使用 CSS 媒体查询 (`@media`)，确保页面在电脑（屏幕宽度大于 768px）和手机（屏幕宽度小于 768px）上都能优雅显示，**避免出现横向滚动条**。
- **任务 C (数据流基础 - JS 交互)：**
 - **目标：**使用原生 JavaScript，实现前端最基本的交互和数据封装。
 - **步骤：**
 1. **获取元素：**在 JS 中，通过 `document.getElementById` 获取输入框和保存按钮的引用。
 2. **添加事件：**为“保存”按钮添加一个 `click` 事件监听器。
 3. **提取数据：**在点击事件触发时，获取输入框的 `value`（即用户输入的梦境文本）。
 4. **封装 JSON：**将获取到的文本封装成一个符合后端要求的 **JSON 对象**，格式为：`{ "content": "这里是梦境文本内容" }`。
 5. **核心产出：**使用 `console.log()` 打印出这个 JSON 字符串。

2. 后端组 (Backend): 搭建服务框架并接收数据

- **任务 A (环境与框架搭建):**

- **目标:** 搭建 Java 开发环境, 并跑通 Spring Boot 框架的最小服务。
- **步骤:**
 1. **环境配置:** 确保电脑已安装 JDK 17+ 和 Maven。
 2. **IDE 准备:** 安装并熟悉 IntelliJ IDEA。
 3. **项目创建:** 使用 Spring Initializr (或 IDEA 自带功能) 创建一个新的 Spring Boot 项目, 选择 Java 和 Maven, 并添加 `Spring web` 依赖。
 4. **验证服务:** 编写一个最简单的 `Controller` 类, 实现 `GET /api/hello` 接口, 返回 `Hello Dream Team` 字符串, 并在浏览器中访问验证。

- **任务 B (核心接口设计):**

- **目标:** 创建一个能够接收前端数据的 RESTful API 接口。
- **步骤:**
 1. **创建控制器:** 在你的项目里创建一个 `DreamController` 类。
 2. **定义 POST 接口:** 定义一个方法, 使用 `@PostMapping("/api/dream/save")` 注解, 使其能接收 HTTP POST 请求。
 3. **数据接收:** 在方法参数中使用 `@RequestBody String dreamJson` 来接收前端发来的整个 JSON 字符串。
 4. **核心产出:** 在方法内部, 使用 `System.out.println` 将接收到的 `dreamJson` 字符串打印到控制台。
 5. **响应:** 返回一个简单的 JSON 响应体 (例如 `{"status": "success", "message": "已接收梦境数据"}`), 表示接收成功。

- **任务 C (数据库预研):**

- **目标:** 为下周连接数据库做准备。
- **步骤:**
 1. **安装:** 在本地电脑安装 MySQL 或使用在线/Docker 容器。
 2. **建表语句:** 编写创建 `dream_log` 表的 SQL 语句。思考清楚字段类型和长度, 比如梦的内容 (`content`) 应该用什么类型 (建议 `TEXT` 或 `VARCHAR(长度足够大)`)。

3. 算法组 (Algorithm): Prompt 设计与 workflow 编排

- **任务 A (平台入门与调研):**

- **目标:** 熟悉 Coze 平台的核心概念 (Agent 和 Workflow)。
- **步骤:**
 1. **注册与登录:** 完成 Coze 平台的账号注册和登录。
 2. **概念学习:** 阅读 Coze 官方文档, 理解什么是 **Agent (机器人)**, 以及 **Workflow (工作流)** 是如何通过连接不同“节点”来处理复杂任务的。

- **任务 B (情绪与关键词提取 Workflow):**

- **目标:** 设计并跑通一个非代码的 LLM 工作流, 实现基础的梦境分析。
- **步骤:**
 1. **创建工作流:** 在 Coze 中创建一个新的 Workflow。

2. **输入节点**：设置一个文本输入节点，命名为 `Dream_Text`。
3. **LLM 节点 (Prompt Engineering)**：连接一个 LLM 节点，并编写**系统提示词 (System Prompt)**。
 - **提示词要求**：要求模型充当“心理分析师”，根据用户输入的梦境文本，**严格按照 JSON 格式**提取以下信息：`情绪`（范围：恐惧、喜悦、平静、焦虑）、`关键词`（3 个）。
 - **示例 Prompt**：请你分析以下梦境文本，并严格以 JSON 格式输出其主要情绪和 3 个核心关键词。{"emotion": "...", "keywords": ["...", "...", "..."]}
4. **核心产出**：运行这个工作流，输入一段测试梦境（如：“我梦见一只大老虎在追我”），观察它是否能稳定输出我们要求的 JSON 格式结果。

- **任务 C (API 准备)**:

- **目标**：了解如何将工作流转化为后端可用的接口。
- **步骤**:
 1. 查找并记录如何将 Coze 的 Agent 或 Workflow 导出为 API 接口。
 2. 了解调用这个 API 需要哪些要素（例如：**App ID**、**API Key/Token**）。

4. 测试组 (Testing): 测试思维与 Postman 入门

- **任务 A (工具与概念入门)**:

- **目标**：理解软件测试的基本概念并安装工具。
- **步骤**:
 1. **安装 Postman**：下载并安装 Postman。
 2. **学习概念**：学习什么是 **黑盒测试**（不看代码，只看功能）和 **接口测试**。理解“测试用例”是用来指导测试行为的文档。
 3. **熟悉界面**：了解 Postman 的基本界面：工作区 (Workspace)、集合 (Collection)、请求 (Request)。

- **任务 B (设计接口测试用例)**:

- **目标**：针对后端组的 `POST /api/dream/save` 接口，设计全面的测试用例。
- **步骤**:
 1. **创建 Collection**：在 Postman 中创建一个新的集合，命名为 `DreamPlatform_API_Tests`。
 2. **创建 Request**：在集合中创建一个新的 POST 请求，请求地址暂定为后端组的本地地址（例如 `http://localhost:8080/api/dream/save`）。
 3. **设计用例**：至少设计以下三种类型的测试用例，并写出“**预期结果**”：
 - **正向用例 (Happy Path)**：
 - **Case 1.1**：提交一段正常长度（例如 500 字以内）的梦境文本。**预期结果**：HTTP 状态码 200，且响应体中包含 `"status": "success"`。
 - **边界用例 (Boundary Case)**：
 - **Case 2.1**：提交一个**超长**的梦境文本（例如 5000 字）。**预期结果**：取决于后端设计，可能是 200 成功接收，也可能是 4xx 错误提示文本过长。
 - **Case 2.2**：提交一个**空**的梦境文本（`"content": ""`）。**预期结果**：可能是 4xx 错误，提示内容不能为空。

- **异常/负向用例 (Negative Case) :**

- *Case 3.1*: 提交一个完全错误的请求体, 例如不传 `content` 字段。**预期结果**: HTTP 状态码 400 (Bad Request)。
- *Case 3.2*: 请求方法错误 (例如用 GET 访问 POST 接口)。**预期结果**: HTTP 状态码 405 (Method Not Allowed)。

- **任务 C (产出):**

- **核心产出**: 将所有设计的用例整理成表格 (包含: 用例名称、测试数据、预期结果), 并准备在下周使用 Postman 对后端组的 API 进行第一次测试。